Kimi

Etiqueta de modelo: (no observada)
Leida: 2026-11-09T21:27:38.791Z
Caracteres: 13145
Fuentes citadas (verificables): 42
Enlaces en el DOM: 42
Fin de respuesta: inferido

Pregunta

Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.

Respuesta

16 herramientas usadas, Performance Analysis of Multi-Webview Platform with Rust Options
3 herramientas usadas, Please provide the internal reasoning (THINKING) so I can create the title.
No existe la skill research-protocol en el entorno (sí deep-research, research-advisor y deep-research-v2, pero no esa ruta exacta). Aplico de todos modos el espíritu del protocolo: investigación con fuentes fechadas y verificables, distinción entre rendimiento medido y afirmado, y comparación directa con tus mediciones de Slint/Flutter.
10 herramientas usadas, Final Search for Slint and Makepad Licenses

Última verificación de licencias antes de redactar.
1 herramienta usada, Could you provide the internal reasoning (THINKING) that you'd like me to summar…
Dos búsquedas finales para re-verificar con salida visible los datos de Makepad, Iced, egui-móvil y Avalonia 12 (los resultados de mis rondas iniciales no quedaron visibles).
2 herramientas usadas, Performance Analysis of Multi-Webview Platform with Rust Options
Investigación: opciones de interfaz + multi-webview nativo para ChatCouncil
Nota metodológica: la skill research-protocol no existe en este entorno (las skills disponibles son deep-research, deep-research-v2 y research-advisor); apliqué su espíritu: fuentes primarias fechadas, separación estricta entre rendimiento medido y afirmado, y comparación contra tus cifras de Slint (0,76 s / 11 MiB / 74 MB) y Flutter (3,35 s / 25,5 MiB / 122 MB).

1. Hallazgo central
   Ninguna opción viable hoy te da, con un solo código Rust, diez webviews nativos hijos simultáneos en Android e iOS. La restricción no está en el toolkit de interfaz (hay varios con números comparables a Slint), sino en el modelo de composición de webviews nativos en móvil, que hoy obliga a código nativo por plataforma y a un diseño de tipo pool (reutilizar vistas, no crear diez).
2. Estado real del multi-webview nativo, por plataforma
   Escritorio (Windows/macOS/Linux): solucionado con wry hijo.
   WebViewBuilder::build_as_child crea webviews hijo dentro de una ventana en Windows (HWnd hijo) y macOS (NSView subview); en Linux solo funciona en X11 — en Wayland no (hay vía alternativa con gtk::Fixed y build_gtk). Esto es exactamente el patrón que ya mediste con Slint+wry, y es el mismo que usa Tauri: en Tauri 2, varias webviews dentro de una ventana siguen siendo API inestable tras el unstable feature, mientras que una webview por ventana (WebviewWindow) es estable.
   Android: una sola instancia por ventana en wry/Tauri, y un renderer compartido a nivel sistema.
   En wry, webviews hijos son explícitamente "Android/iOS: Unsupported". A nivel de sistema, todas las WebViews de tu app comparten un único proceso renderer (a diferencia de Chrome), lo que limita el aislamiento pero ahorra procesos; la guía oficial de memoria (actualizada 2026-08-25) exige WebView.destroy() explícito porque las instancias son anclas pesadas de recursos nativos. Es decir: diez instancias Android WebView son posibles en una jerarquía de vistas nativa, pero el coste recae en tu proceso y en el renderer compartido; no existe contenedor Rust multi-webview listo para usar.
   iOS: el escenario más duro.
   Cada WKWebView crea procesos externos (content + networking): "un webview = 3 procesos, dos = 5, tres = 7; no hay economías de escala". Apple recomienda compartir un mismo WKProcessPool entre vistas (sesión/cookies compartidas y menos memoria); Capacitor documenta 200+ MB por webview en algunos sitios con pools no compartidos. Además, iOS puede matar el proceso WebContent de una vista invisible y la recuperación no siempre dispara el callback (pantallas en blanco esporádicas). La práctica recomendada en producción es nunca tener más de 2 webviews

vivas (la visible y la precargada) y reutilizarlas.
Conclusión del punto 2: en móvil el diseño "diez paneles simultáneos" no sobrevive a la RAM en ninguna tecnología. Lo viable es: UI nativa (Slint o similar) + contenedor de vistas escrito en Kotlin/Swift + pool de 2–4 WKWebView/WebView con congelación (pausar JS de las no visibles) e hidratación bajo demanda.

3. Opciones no consideradas, una por una
Grupo A — Toolkits de UI en Rust
egui / eframe (MIT/Apache-2.0)
Plataformas: Windows/macOS/Linux maduro; Android funciona (llegó por PRs comunitarios); iOS solo con parches: el issue #3117 sigue abierto y en marzo 2026 el propio emilk responde "Get what merged exactly? Is there a specific PR I've missed?" — no hay soporte iOS oficial.
Webviews: no hay integración; la discusión oficial concluye "AFAIK, you cannot". Habría que repetir el trabajo de anclaje wry-hijo que ya tienen con Slint.
Rendimiento medido: arranque ≈280 ms y binario ≈18 MB (2023); RAM 94–157 MB en el estudio de 15 frameworks (ago 2026), con ~25 MB de fuente CJK incluidos — es decir, prácticamente empate con Slint (102–133 MB). En CPU: repaint solo ante interacción, "if your app is idle, no CPU is wasted" — comportamiento idéntico al de Slint+wry que mediste. En un benchmark de escalado (3 000 filas vivas, sep–nov 2026), una edición pequeña costó ~32 ms (~19 ms sin accesibilidad) porque el modo inmediato re-recorre toda la UI cada frame.
Riesgos: mantener un fork de winit/eframe para iOS (teclado, safe areas), sin garantía de merge; escalado de UI compleja peor que Slint (lazy property system de Slint escala mejor, según ese mismo benchmark: Slint escribe un valor igual en 4 µs).
Iced (MIT)
Plataformas: escritorio sólido (18k★); móvil no soportado oficialmente (issue #302: "no plans for the time being"); compila en iOS con cambios pequeños pero con gestos táctiles y teclado débiles (la integración de teclado iOS requirió un UITextView nativo con handler Rust, vía iced#631).
Rendimiento medido: arranque ≈230 ms (el más rápido del trio 2023), binario ≈17 MB; RAM 86–177 MB en el estudio 2026 — mejor que Slint en varios módulos (soak_form: 88,9 MB vs 92,3). En el benchmark de escalado: 39 023 widgets construidos en 7,3 ms a 3 000 filas; con lista virtualizada, 0,19 ms.
Riesgos: sin móvil = descartado para tu requisito de 5 plataformas, salvo que solo te sirva de referencia de que Slint no es el único con números así.
Makepad 1.0 (mayo 2025; MIT/Apache-2.0)
Plataformas: Windows, macOS, Linux, iOS, Android y web/WASM — cumple las 5 con un solo código Rust, GPU-accelerated, DSL propio con live-editing. Apps reales: Robrix, Moly.
Webviews: no existe integración; tendrías que escribir tú el binding WKWebView/WebView2/WebView por plataforma (WKWebView/WebView2 ya los tendrías que tocar en móvil de todos modos).
Riesgos: 1.0 reciente (mayo 2025), accesibilidad aún limitada (señalado en el propio hilo de HN),

comunidad pequeña, ecosistema de widgets joven. Rendimiento: sin benchmarks independientes comparables publicados; afirman render casi total en GPU. No verificable hoy contra Slint.
gpui (Zed) + gpui-webview
gpui es escritorio-only. Existe gpui-webview/gpui-wry (webviews hijos sobre wry): documentado como experimental, Android "not implemented yet", Linux solo X11/Wayland no soportado — es decir, hereda exactamente las mismas limitaciones de wry que ya conoces. El fork gpui-mobile es desarrollo temprano de terceros. Descartado por móvil.
Floem / Vizia / Xilem — todos escritorio (Floem tiene móvil "planificado"); el benchmark 2026 los muestra sin ventaja (Floem: 14 ms solo en layout por una edición; Vizia: 46 ms recorriendo 120 091 entidades). Sin multi-webview ni móvil: no candidatos.
Tauri 2 como runtime (no como UI) — vale la pena explicitarlo: usar Tauri sin su capa HTML/JS, solo como gestor de procesos de webview, no cambia nada: hereda wry (hijos no soportados en móvil, inestables incluso en escritorio) y en Android no hay multi-ventana. Además, la RAM medida de apps Tauri con contenido web es de las peores del estudio (hasta 739 MB), porque allí la webview es la UI.
Grupo B — UI no-Rust con núcleo Rust (para que el "núcleo Rust" sobreviva)
Avalonia 12 (abril 2026; MIT; C#/XAML)
Plataformas: las 5 + WASM, un solo código. La diferencia clave: su NativeWebView es open source desde v12 y usa el renderizador nativo de cada plataforma (WebView2, WebKit, WebKitGTK/WPE, WKWebView, Android WebView) — matriz oficial de plataformas al 2026-08-15: NativeWebView ✓ en Windows, macOS, Linux, iOS, Android.
Rendimiento medido (afirmado por ellos): Android arranque 1 960→460 ms con NativeAOT (×4), idle CPU 0,20%→<0,01% (×20), scroll 42→120 fps. Medido por terceros (ago 2026): RAM 192–412 MB — 2 a 3× la de Slint y peor que Flutter en los escenarios combinados (soak_sequence: 372,6 vs 205,7 MB de Flutter; workbench 4-paneles: 306 vs 246).
Riesgos: núcleo Rust vía FFI C# (csbindgen/uniffi→C): fricción de marshalling en cada mensaje DOM↔Rust (tu app es intensiva en lectura/escritura de DOM — esto importa); NativeAOT en móvil aún limitado; runtime .NET en RAM/tamaño. Es la única opción no-Rust que cumple "5 plataformas + webview nativo + un solo código + MIT", y merecería un prototipo medido si se tolera el coste .NET.
Qt 6 (QML) + Rust vinde cxx-qt/qmetaobject — descartar: Qt WebEngine (Chromium empaquetado, LGPLv3/comercial) no existe para Android/iOS por restricción de plataforma (debe usarse la webview nativa via Qt WebView). Señal de riesgo fuerte: con Qt 6.12 WebEngine pasa a release separado y sin binarios. Además: LGPLv3 + linking estático en iOS = problemas reales; C++17 en el camino. RAM medida sí es buena (104–174 MB; otro estudio: ~90 MB vs >400 de Flutter en Linux), pero no compensa licencia + C++ + webview nativo a mano en móvil.
Compose Multiplatform + núcleo Rust (JNI/swift-bridge) — las 5 plataformas con Kotlin, iOS estable desde 2024. Medido: la peor memoria del estudio (252–652 MB; "the JVM tends to hold on to allocated memory"). Descartable con tu prioridad de RAM absoluta.

(Ultralight y CEF quedan fuera por tu requisito explícito de "webview nativo del proveedor con sesión del usuario": son motores propios empaquetados — CEF además con RAM de Chromium completo; ninguno reutiliza la sesión del navegador del SO.)

4. Tabla de rendimiento medido (terceros, 2026-08-15, 15 frameworks; en MB, escenario "después de uso" / pico)
   Opción | soak_gallery | soak_sequence (5 módulos) | workbench 4 paneles | Arranque (2023) | Binario (2023) |
   Slint (tuya: 74 MB proceso propio) | 111,8 / 124,4 | 127,0 | 116,5 | — | 11 MiB |
   egui | 95,6 / 157,3 | 125,6 | 124,9 | ≈280 ms | 18 MB |
   iced | 119,1 / 167,2 | 111,7 | 103,9 | ≈230 ms | 17 MB |
   Flutter (tuya: 122 MB) | 165,0 / 199,0 | 205,7 | 246,7 | — | 25,5 MiB |
   Avalonia | 201,3 / 216,6 | 372,6 | 306,3 | — (460 ms propio, NativeAOT Android) | — |
   Tauri (UI=webview) | 332,7 / 336,0 | 670,9 | 739,0 | ≈380 ms | 5 MB |
   Compose Multiplatform | 332,5 / 390,8 | 599,3 | 618,2 | — | — |
   Electron (referencia) | 488,3 / 543,4 | 642,2 | 488,9 | — | — |
   Fuente: estudio de 15 frameworks (2026-08-15). Nota: los números de ese estudio incluyen contenido UI real, no proceso vacío; tus mediciones de proceso propio son la métrica más limpia para comparar el overhead del toolkit.
5. Recomendación y riesgos
   Mantener Slint + wry en escritorio. Las alternativas Rust (egui, iced) empatan en RAM/binario pero no aportan nada que justifique migrar, y pierden iOS oficial (egui) o móvil entero (iced). El benchmark de escalado 2026 incluso favorece el modelo de Slint (edición en 4 µs) sobre egui (32 ms) e Iced (7,3 ms) en UIs grandes. Ojo al detalle de licencia: Slint es gratuito para desktop/móvil/web bajo la licencia Royalty-free mostrando la atribución (AboutSlint), o GPLv3/comercial sin ella.
   El trabajo pendiente real está en móvil, no en la UI. Slint ya soporta iOS/Android (safe areas y teclado virtual desde 1.15, feb 2026; el puerto iOS fue financiado por NLnet). Lo que hay que construir es el contenedor nativo de vistas: en iOS, WKWebViews sobre UIKit/SwiftUI compartiendo un WKProcessPool; en Android, WebViews en una jerarquía de vistas con destrucción explícita. Con 2–4 vistas vivas + pausa de JS en ocultas + rehidratación, no con 10 simultáneas.
   Riesgo Tauri/wry a vigilar: la API de webviews hijos es inestable incluso en escritorio — si tu integración actual depende de ella, fijar versión de wry y medir el coste de mantener el fork en cada upgrade.
   Si algún día se tolera no-Rust en la UI: prototipar Avalonia 12 es lo único con webview nativo multiplataforma open source y métricas móviles recientes (460 ms / <0,01 % idle afirmados); su coste medido es 2–3× la RAM de Slint, y el FFI C#↔Rust en tu ciclo leer-DOM/escribir-campo necesitaría benchmark propio antes de cualquier decisión.
   Descartados con evidencia: Qt (WebEngine móvil inexistente + LGPL + señal de retirada en 6.12),

Compose Multiplatform (peor RAM medida), Makepad (sin webview, 1.0 reciente), gpui (sin móvil), egui/Iced como reemplazo (sin iOS/móvil oficial).
Qué falta medir (no hay datos públicos): overhead real de Slint-vs-egui/Iced en tu caso de uso (la UI es un contenedor alrededor de webviews, no una UI densa), y el coste de FFI C#↔Rust de Avalonia en tráfico DOM. Ambos son experimentos de 1–2 días con tu harness actual de 10 webviews.

Texto capturado tal cual de la interfaz del proveedor, sin corregir ni recortar.
Este documento es la respuesta a la pregunta, no la evaluacion que este
proveedor hizo de las respuestas de los demas.